



AutoGrader+

Your repository, marked like a code review

Multi-agent grading of full-stack projects, and a learning platform around it

Project report

Live application: <https://autograder-plus.onrender.com>

Source code: <https://github.com/roshanraj9136/auto-grader>

CSL100 group project, October 2026

Contents

1. Summary	10. Security
2. Problem and goals	11. Student experience
3. System overview	12. Instructor experience
4. Architecture	13. Labs
5. Grading pipeline	14. Data model
6. Multi-agent design	15. Deployment and operations
7. Scoring	16. API
8. Latency	17. Limitations and future work
9. Reliability and scaling	

1 Summary

AutoGrader+ grades a **whole full-stack project** from its GitHub URL and returns an explainable report in about a minute. Five specialist reviewers (code quality, architecture, security, testing, Docker & DevOps) examine the repository **in parallel**; a judge agent reads their five reports, calibrates the scores within a fixed bound and writes the verdict, the top priorities and a learning path. The overall score is computed **in code** from the assignment's published rubric weights, so grades are reproducible.

Around the grader we built a learning platform with two role-specific views. Students get assignments with visible rubrics, live grading progress, detailed feedback, a progress dashboard and hands-on labs. Instructors get a class overview that starts with what needs attention, a gradebook, per-assignment analytics with copy detection, student profiles, grade adjustments with written reasons, and re-grading. The system runs as stateless API replicas behind a load balancer with PostgreSQL, and is deployed on Render with Neon.

6 agents per grading: 5 reviewers, 1 judge	2.74× faster than the same stages in sequence	1.2 s re-grade of unchanged code (cache hit)	44 automated security and access checks, all passing
--	---	--	--

2 Problem and goals

Grading full-stack projects by hand is slow, because each submission combines a frontend, an API, a database and container configuration; it is inconsistent, because different graders weigh issues differently; and feedback usually arrives after the deadline, when it no longer helps. Instructors also lack an overview of who is struggling and whether students copied each other's work.

Goal	How the design meets it
Fast feedback	Parallel pipeline (DAG), speculative clone, result cache, small ranked prompts, live progress over Server-Sent Events.
Bounded worst case	A deadline on every stage; a reviewer that misses its deadline falls back to a deterministic scorer.
Reproducible, explainable grades	Total computed in code from published weights; the judge may move each area by at most ± 1.5 and must explain why.
Safe on untrusted code	GitHub-only URL allow-list, no shell, secrets redacted before prompts, locked-down optional sandbox.
Useful to students	Priorities with file names, fixes, a learning path, resubmission with the best attempt counting.
Useful to instructors	Attention list, gradebook, assignment statistics, copy detection, grade adjustment, re-grading.
Works without an API key	Every reviewer has a rule-based scorer; the same pipeline and report format are used (heuristic mode).

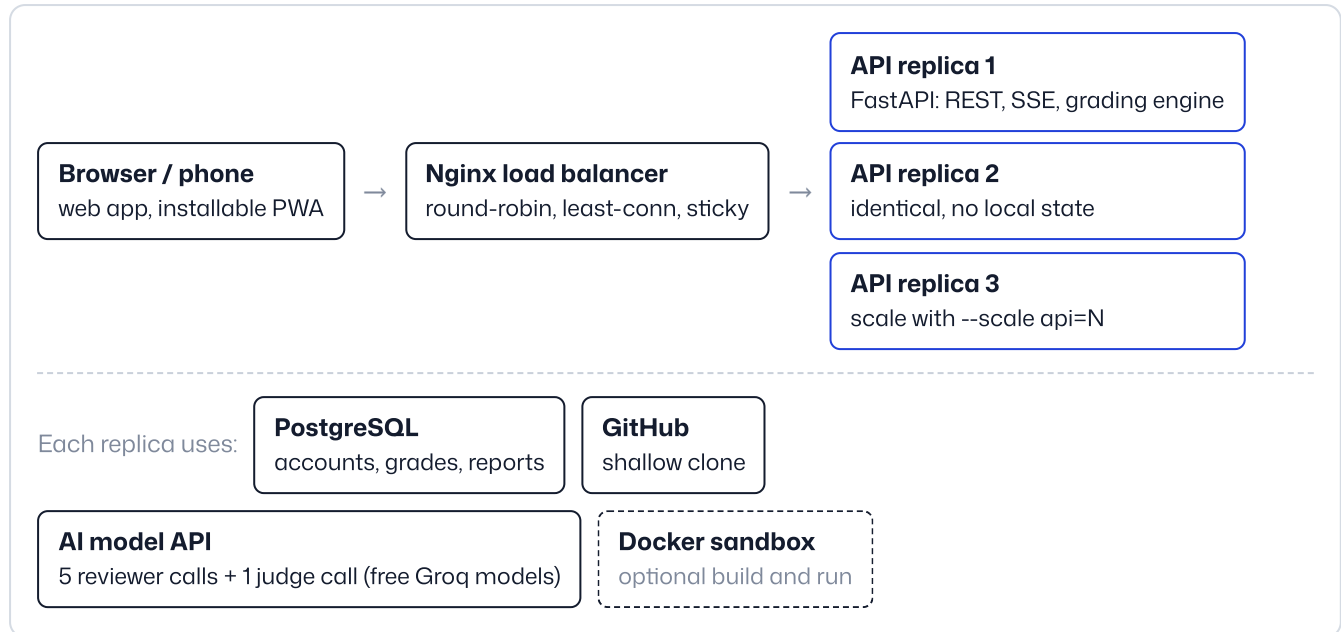
3 System overview

The system has three parts. The **grading engine** turns a repository into a report. The **student view** is built around one question, "what should I do next?". The **instructor view** is built for a desk: dense tables, and the problems that need attention first.

Area	Reviewer checks
Code quality	readability, file size, duplication, comments, linting
Architecture	layering, coupling, module dependency graph and cycles, API and database design, scalability
Security	OWASP Top 10 issues, hard-coded secrets, risky patterns, authentication and authorisation code
Testing	unit, integration and end-to-end tests, test-to-source ratio, CI that runs the tests
Docker & DevOps	Dockerfile quality (20+ lint rules), multi-stage builds, non-root users, compose files, CI/CD

4 Architecture

The application is a set of **stateless API replicas** behind an Nginx load balancer, sharing one PostgreSQL database. Each replica serves the web app, the REST API and the live progress streams, and runs grading jobs.



- **Shared state lives in PostgreSQL:** users, sessions, assignments, submissions, grade adjustments, reports, lab progress, replica heartbeats. Any replica can serve any page.
- **Live progress is durable:** a job runs in the replica that accepted it and fans out its events in memory, and one writer task per replica also appends every event, in order, to the `job_events` table. A replica that does not hold the job streams it from that table, so any replica can serve any job's progress and sticky routing is no longer required.
- **Locally**, `docker compose up` starts Nginx, three API replicas and PostgreSQL on a private network. **In production** one container runs on Render with a Neon PostgreSQL database; the code is the same.

Layers in the code

Layer	Responsibility	Code
Web app	Single-page app in plain JavaScript modules: role-specific layouts, live progress, SVG charts, PWA	<code>web/</code>
HTTP API	Validation, sessions, REST, Server-Sent Events, report downloads, security headers	<code>app/main.py</code>
Platform	Accounts, assignments, dashboards, instructor analytics, gradebook, labs, demo class	<code>app/platform/</code>
Grading engine	Job queue with admission control and de-duplication, the pipeline DAG, tracing, metrics	<code>app/jobs.py</code> , <code>app/pipeline.py</code>
Agents	Five specialists and the judge, structured output, heuristic fallbacks	<code>app/agents/</code>
Ingest, sandbox	Git, one-pass repository index, evidence selection, Dockerfile linter, sandbox	<code>app/ingest/</code> , <code>app/sandbox/</code>
Reports	HTML / Markdown / JSON reports, content-addressed result cache	<code>app/report/</code>

5 Grading pipeline

A submission becomes a **job**, which runs as a dependency graph so that independent stages overlap.

1. **Admission.** A semaphore limits concurrent jobs and reports a queue position. An identical in-flight request attaches to the running job (single-flight).
2. **Resolve and clone in parallel.** `git ls-remote` finds the exact commit while a shallow, single-branch, tag-less clone starts speculatively.
3. **Cache lookup.** The same commit, Dockerfile, rubric and models return the stored report (about 1.2 s); the clone is cancelled and its process tree killed.
4. **Index.** One pass over the repository (about 35 ms for an 8-service repository) builds a read-only index: languages, file sizes, tests, CI, Dockerfiles, the module dependency graph, and a secret scan whose matches are redacted.
5. **Docker.** A static linter with 20+ rules always runs. A locked-down build and smoke run is optional.
6. **Five reviewers in parallel.** Each receives the evidence relevant to its area within a character budget (about 3.6k to 7.2k tokens) and must answer through forced tool use with a JSON schema.
7. **Judge.** Reads only the five compact reports (about 1k tokens), may move each area by at most ± 1.5 with a rationale, and writes the summary, priorities and learning path.
8. **Report.** The final score is computed in code; HTML, Markdown and JSON reports are written atomically to disk and copied into PostgreSQL; the browser receives the final event.

Critical path

$$T_{\text{total}} \approx \max(T_{\text{resolve}}, T_{\text{clone}}) + \max(T_{\text{index}} + \max(T_{\text{reviewer 1-4}}), T_{\text{docker}} + T_{\text{devops}}) + T_{\text{judge}}$$

Only the DevOps reviewer waits for Docker; the other four start as soon as indexing finishes, so a slow build overlaps with model calls. Every report records its measured critical path (found by walking back from the report along the dependency that finished last) and the speed-up $T_{\text{sequential}} / T_{\text{total}}$.

6 Multi-agent design

Agent	Evidence it receives (selected from the shared index)
Code Quality	largest files and a diverse sample across modules, duplication and comment metrics
Architecture	entrypoints, routes / services / models, module dependency graph and cycles, compose, nginx, k8s files
Security	secret-scanner and risky-pattern hits (as leads to verify), authentication files, .gitignore
Testing	test files, test-to-source ratio, CI workflows, test scripts
DevOps & Docker	Dockerfile, lint findings, build and run result and logs, compose, CI
Judge	only the five structured reports, never raw code

- **Structured output.** Every call uses forced tool use with a JSON schema; nothing is parsed from free text. Scores are clamped and validated in code. Findings are capped at seven per reviewer.
- **Bounded judge.** The judge calibrates rather than re-grades: each area can move by at most ± 1.5 and every change needs a rationale.
- **Graceful degradation.** If a model call fails or misses its deadline, only that reviewer falls back to its rule-based scorer, and the report marks it.
- **Prompt layout.** The tool definitions and the shared repository context are byte-identical across the five reviewer prompts, so the provider's prompt cache can reuse them on re-grades.

7 Scoring

`final score = 10 × ∑ (weight_area × score_area)`, with weights summing to 1 and area scores from 0 to 10

The formula runs in code, so the same area scores always give the same total and every grade can be audited. Weights are set per assignment and shown to students before they submit; the defaults are code quality 25%, architecture 25%, testing 20%, security 15% and Docker & DevOps 15%. A student may resubmit as often as they like and the **best** attempt counts.

Grade	A	A-	B	B-	C	C-	D	F
Score from	85	78	70	62	55	48	40	0

Instructor adjustments

An instructor can set a graded submission's score by hand. A reason is mandatory; the original score, the new score, the reason, the instructor and the time are stored in `grade_overrides`. The student sees the adjusted score and the reason on their feedback page, and the instructor can restore the original at any time. Adjusted rows are protected from being overwritten if the job's state is ever replayed.

8 Latency

Technique	Effect
Fan-out of 5 reviewers	the model phase costs the slowest reviewer, not the sum of five
Docker overlapped with reviewers	the build leaves the critical path unless it is the longest branch
Speculative clone ls-remote	a cache miss pays nothing for resolution; on a hit the clone is cancelled
Content-addressed result cache	unchanged code with an unchanged rubric is answered in about 1.2 s
Single-flight de-duplication	identical in-flight requests share one job
Shallow, single-branch, tag-less clone	minimum objects transferred
One-pass indexer	35 ms for an 8-service repository; no reviewer re-reads the disk
Ranked evidence with budgets	3.6k-7.2k input tokens per reviewer keeps prefill short
Capped output, concise schema	decoding dominates model latency, so output is kept short
Small judge input	the only sequential model call reads about 1k tokens
Global model-call semaphore	bursts queue locally instead of causing rate-limit retry storms
Non-blocking I/O	git, Docker and indexing run in worker threads; live streams keep flowing

Benchmark

`scripts/latency_benchmark.py` on `dockersamples/example-voting-app`: real clone, index, lint, evidence and judge logic; model latency simulated as 0.8 s to first token, 20k input and 70 output tokens per second, so the figures isolate the

architecture's effect.

Metric	Value
Wall-clock, 5 reviewers + judge	27.5 s
Same stages run sequentially	75.5 s
Speed-up from the DAG	2.74×
Measured critical path	clone → index → security → judge → report
Cache-hit re-grade (real)	1.2 s
Index (real)	34–38 ms

Stage	Typical	Hard limit
Resolve and clone	0.8–2.5 s	120 s
Index	< 0.1 s	5,000 files, 40 MB read
Each reviewer	10–25 s	120 s, then rule-based fallback
Docker build and smoke run	20–300 s	600 s + 8 s run
Judge	8–15 s	150 s, then rule-based calibration

GET /api/metrics returns rolling p50, p95 and maximum latency per stage, shown live on the Platform health page, so tail latency is observed rather than guessed.

9 Reliability and scaling

- **Deadlines** on git, Docker, every model call and the judge; the SDK retries 429 and 5xx responses with exponential backoff.
- **Admission control** with a FIFO queue and a visible position; failed or cancelled jobs cancel their background work and always delete their workspace.
- **Heartbeats.** Every replica writes a heartbeat every 20 s. Any replica fails the unfinished submissions of a replica that stopped (crash, scale-down, recreated container), removes its orphaned clone directories, and repairs its own rows if a state update was lost.
- **Durable reports.** Artifacts are written atomically and copied into PostgreSQL, so they survive free hosts whose disk is wiped on restart, and replicas without a shared volume.
- **Database pool** of at most 10 connections per replica, pinged before reuse; schema creation and seeding are serialised with a PostgreSQL advisory lock so replicas can start together.
- **Replayable streams.** Because events are stored, a stream continues on any replica; if the replica running a job dies, the stream ends with the saved outcome once peers mark the submission failed. A two-replica test on one database confirmed that the second replica replays exactly the same 33-event sequence.

Scaling path

Today the web tier scales by adding replicas. The next steps follow how large course autograders are built; Autolab's grader (Tango) splits a job queue from a job manager that hands jobs to free workers, and runs 2,000+ daily submissions across six grading servers at the University at Buffalo.

1. **A durable work queue in PostgreSQL.** Workers claim jobs with `UPDATE ... WHERE id = (SELECT id ... FOR UPDATE SKIP LOCKED LIMIT 1)`; many workers drain one table without blocking each other, jobs survive restarts, and no new infrastructure is needed (sound up to a few thousand jobs per second).
2. **Separate web and worker roles** from the same image, scaled independently.
3. **Isolated build workers** (BuildKit, or Kubernetes Jobs with gVisor or Kata), one fresh environment per submission.
4. **Object storage** (S3 or MinIO) for reports; **per-area retries** instead of re-running a whole job.
5. **Courses**, so one deployment serves many classes, and **shared rate limits** in Redis across replicas.

10 Security

Threat	Defence
Malicious repository input	Only <code>https://github.com/owner/repo</code> is accepted (blocks <code>file://</code> , <code>ext::</code> , SSH and SSRF); refs are validated and cannot start with <code>'</code> ; every subprocess runs with an argument list, never a shell; git never prompts for credentials.
Secrets in student code	Nine scanner rules run during indexing; matched values are redacted before any prompt or report.
Untrusted Docker builds	Containers run with <code>--network none</code> , 512 MB, 1 CPU, 256 processes, all capabilities dropped and <code>no-new-privileges</code> ; the sandbox is opt-in.
Stolen database	script password hashes with per-user salts; 256-bit session tokens stored only as SHA-256, so a leaked database cannot be replayed as sessions.
Students changing grades	Every instructor endpoint checks the role on the server. No student endpoint writes a score. Students can read only their own submissions and reports.
Password and join-code guessing	Sign-in is throttled per IP (10 failures / 15 min) and per account (100 / 15 min, so nobody can lock the instructor out); wrong join codes per IP and in total; comparisons are constant-time; unknown emails take as long to reject as wrong passwords.
Forged client addresses	Throttles use an address the client cannot write: Cloudflare's CF-Connecting-IP on Render, X-Real-IP behind the bundled Nginx, otherwise the right-most X-Forwarded-For hop. A live test showed Render forwards the client's own X-Forwarded-For value first, which is why the left-most hop is never trusted.
Public demo misuse	Only a demo student login is public; the instructor side needs the instructor's own account. Sign-up is closed in demo mode; demo logins cannot change their password or name, and the server refuses them every instructor action.
Grade integrity	An adjusted grade is pinned as the assignment grade in every view, even over later or higher attempts, until restored; re-runs are refused when the original inputs (custom rubric, uploaded Dockerfile) are not stored.
Cross-site attacks	HttpOnly, SameSite=Lax, Secure cookies; state-changing API calls from other sites are rejected (Fetch Metadata and Origin checks); strict Content-Security-Policy; HSTS; framing by other sites is blocked; the frontend lab runs student code in a sandboxed iframe.
Spreadsheet injection	The CSV gradebook neutralises cells that start with <code>=</code> , <code>+</code> , <code>-</code> or <code>@</code> .

Verification

An automated script (`scripts/security_check.py`) signs in as an anonymous visitor, the demo student and the real instructor and runs 44 checks, including: students and visitors are refused every instructor endpoint; students cannot adjust, re-grade, create or delete; the retired demo instructor login no longer works; adjustments are validated (0–100, reason required) and reversible; students cannot read other students' reports; reserved emails cannot be registered; brute-force sign-in is throttled; cross-site requests are blocked; a lowered grade sticks over a higher attempt; practice runs cannot be re-run without their

rubric; forged X-Forwarded-For headers do not reset the sign-in throttle; security headers are present. All 44 pass.

11 Student experience

The student dashboard opens with **Up next** (the nearest unsubmitted assignment, or the one with most room to improve) and the **last result** with the first three things to fix, followed by the average score, assignments and lab tasks done, class rank, score history, a five-area skill profile, deadlines and the learning path.

The screenshot shows the AutoGrader+ student dashboard for a demo student. The interface is divided into a left sidebar and a main content area.

Left Sidebar:

- AutoGrader+ Full-stack course
- Home
- Assignments
- Labs
- Practice
- Your progress
- My submissions
- Leaderboard
- Profile

Main Content Area:

- Greeting:** "Good afternoon, Demo" with a note "3 assignments are waiting for a first submission."
- Level Progress:** Level 1 (58 XP, 92 more to level 2)
- Up next:** "Lab 2 - REST API + PostgreSQL" (Due Oct 22, 11:59 PM, in 13 days). Description: "Design a CRUD REST API (any language) backed by PostgreSQL. Separate routes, services and data access, and test it against a real te...". Checklist: Layered structure (routes -> services -> repositories), Schema migrations, Parameterised queries only, and 2 more on the assignment page. Button: "Start this assignment".
- Your last result:** 58 /100 C. Practice run, 5 hours ago. Feedback items: 1. Remove hardcoded database credentials and use environment variables or secrets management. 2. Disable Flask debug mode ('debug=True') in the production configuration of the vote service. 3. Implement a robust automated testing suite (unit and integration tests) and integrate it into your GitHub Actions CI/CD pipelines. Button: "Read the full feedback".
- Summary Cards:**
 - Average score: 58 (best try on each assignment)
 - Assignments done: 1 of 4
 - Lab tasks: 0 of 17
 - Class rank: #4 (See the leaderboard)
- Score history:** 5 graded, 100 (out of 100)
- Skills:** out of 10

Student dashboard

Each assignment shows its brief, the checklist that will be checked and the weight of each area. After submitting, the student watches the four stages and the five reviewers live. The feedback page shows the score and grade, marks by area with their weights, the numbered priorities with file names, topics to learn, and every finding with its fix; an instructor's adjustment and reason appear at the top.

AutoGrader+

Full-stack course

Home

Assignments

Labs

Practice

Your progress

My submissions

Leaderboard

Profile

DS

Demo Student

Student,
2024CS10001

My submissions

Feedback

58

/100

Grade C

Good progress

dockersamples/example-voting-app

Your project demonstrates a highly impressive distributed microservices architecture with clear service boundaries, multi-language implementations, and solid Docker/Kubernetes configurations. However, the system's overall quality is held back by critical security vulnerabilities (such as hardcoded credentials and Flask debug mode enabled) and a near-total absence of automated testing. Additionally, the reliance on tight polling loops instead of event-driven communication introduces unnecessary performance bottlenecks.

Printable report

Download as Markdown

Marks by area

out of 10, weighted by the rubric

<> Code quality Readable, well-structured code	counts 25%		6.5 /10
Architecture How the app is organised	counts 25%		8.0 /10
Testing Tests that prove it works	counts 20%		2.0 /10
Security Keeping users and data safe	counts 15%		4.0 /10

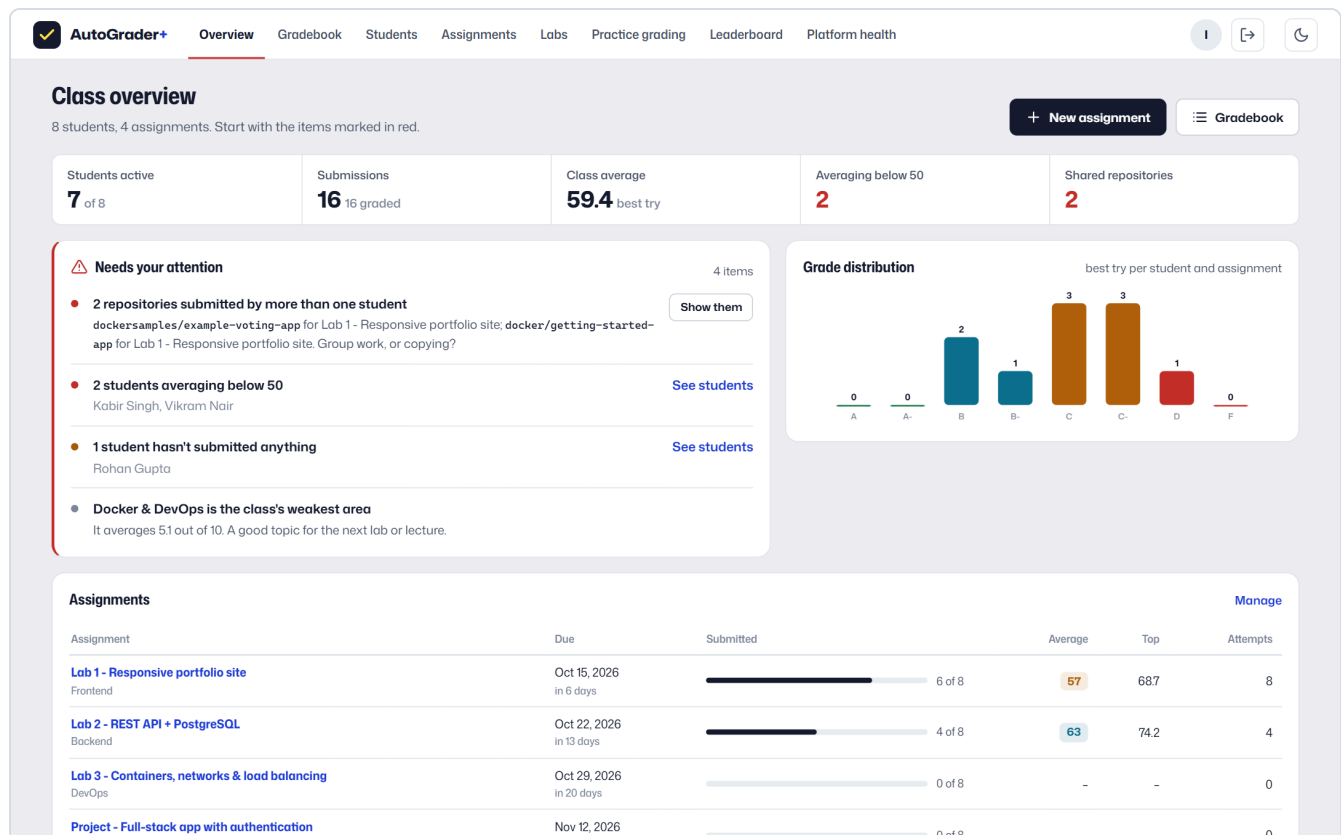
Feedback for one submission

AutoGrader+ project report

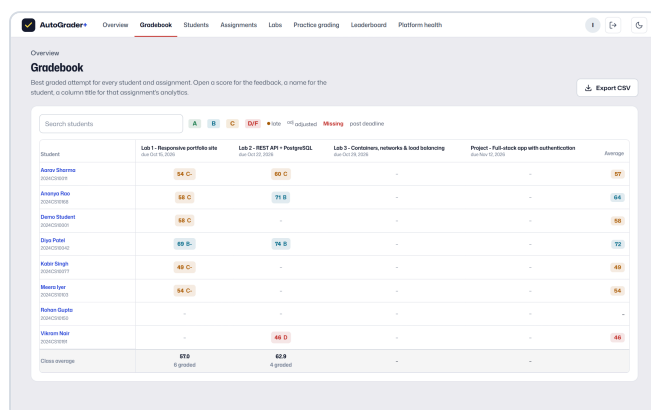
14 / 21

12 Instructor experience

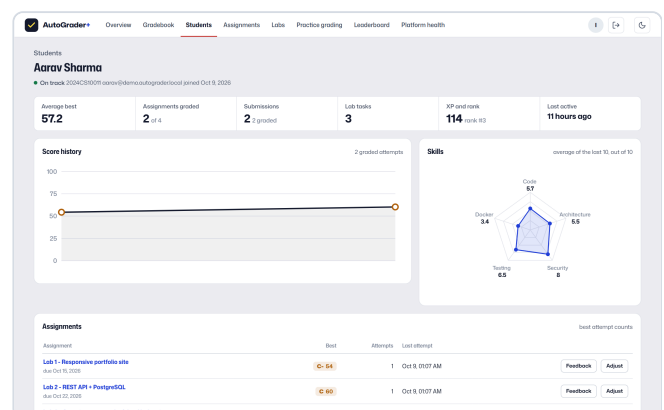
The instructor workspace uses top tabs and dense, condensed tables. The **overview** starts with **Needs your attention**: repositories submitted by more than one student, gradings that failed, students averaging below 50, students who have not submitted anything, deadlines within a week with low completion, and the class's weakest area. Below are the class numbers, the grade distribution, the assignment table and the latest submissions with filters.



Instructor overview



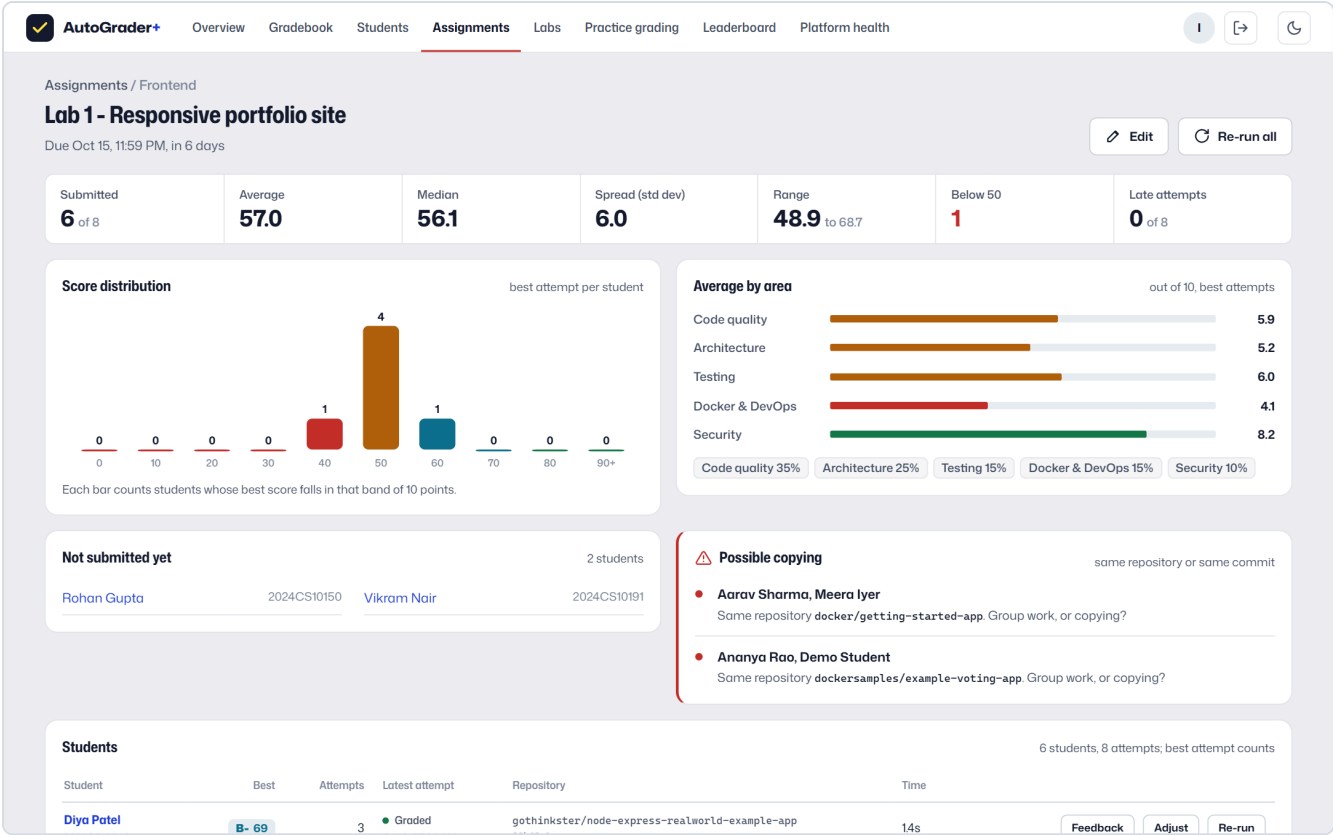
Gradebook: every student and assignment, with late, adjusted and missing marks



Student profile: history, skills, attempts and labs

Assignment analytics

Each assignment has its own page: submitted count, mean, median, standard deviation, range, students below 50 and late attempts; the score distribution in bands of ten; averages per area; submissions per day; the list of students who have not submitted; and **possible copying**, detected when two students submit the same repository, or the same commit in different repositories (an unchanged fork). From the student table the instructor opens feedback, **adjusts a grade** with a reason, or **re-runs** a submission; "Re-run all" grades every student's latest submission again after a rubric change.



AutoGrader+

OverviewGradebookStudentsAssignmentsLabsPractice gradingLeaderboardPlatform health

Assignments / Frontend

Lab 1 - Responsive portfolio site

Due Oct 15, 11:59 PM, in 6 days

EditRe-run all

Submitted6 of 8

Average57.0

Median56.1

Spread (std dev)6.0

Range48.9 to 68.7

Below 501

Late attempts0 of 8

Score distribution

Each bar counts students whose best score falls in that band of 10 points

Not submitted yet

Rohan Gupta2024CS10150Vikram Nair

Adjust grade

Diya Patel, currently 68.7 out of 100. The new score becomes the student's grade for this assignment, even if they submit again, until you restore the original. They see it with your reason on their feedback page.

New score (0 to 100)

68.7

Reason

e.g. Tests run in CI but the grader could not detect them

Save scoreCancel

Testing started app. Group work, or copying?

Ananya Rao, Demo Student

Same repository dockersamples/example-voting-app. Group work, or copying?

Students

6 students, 8 attempts; best attempt counts

Student	Best	Attempts	Latest attempt	Repository	Time	
Diya Patel	B- 69	3	Graded Oct 9, 07:09 AM	getthinkster/node-express-realworld-example-app	14s	FeedbackAdjustRe-run

Adjusting a grade: a reason is required and the original can be restored

13 Labs

Lab	What students practise	How it is checked
Frontend	HTML, CSS and JS editor with a live preview in an opaque-origin sandboxed iframe	DOM checks in the sandbox
Databases	SQL joins, GROUP BY, and an index that turns a table scan into a search	server runs the query on a fresh read-only in-memory copy and compares with a reference (XP)
Load balancers	round-robin, least-connections and sticky routing across real replicas	real requests through Nginx; replica identity in X-Served-By
Networks	round-trip time and jitter, proxy headers, Server-Timing breakdown	Resource Timing API and the backend's Server-Timing header
Docker	fix a Dockerfile until the linter is clean, multi-stage, non-root with a healthcheck	the grader's own 20+ rule linter on the server (XP)

Only tasks verified by the server earn XP, so the leaderboard cannot be inflated with forged browser calls.


AutoGrader+
Full-stack course

[Home](#)
[Assignments](#)
[Labs](#)
[Practice](#)

Your progress

[My submissions](#)
[Leaderboard](#)
[Profile](#)

DS **Demo Student**
Student, 2024CS10001

Labs / Database

Databases: SQL & indexes

Query a real relational schema (students, courses, enrollments) and read query plans.

Query editor
Ctrl + Enter to run

Task to check (optional)

Free play: no check

```
SELECT * FROM courses;
```

Run query

☐ Index on enrollments(student_id)

Show a solution

Tables

```
CREATE TABLE students (id INTEGER PRIMARY KEY, name TEXT NOT NULL, branch TEXT NOT NULL, year INTEGER NOT NULL);
CREATE TABLE courses (id INTEGER PRIMARY KEY, code TEXT NOT NULL UNIQUE, title TEXT NOT NULL, credits INTEGER NOT NULL);
CREATE TABLE enrollments (student_id INTEGER NOT NULL REFERENCES students(id), course_id INTEGER NOT NULL REFERENCES courses(id), semester TEXT NOT NULL, grade_points INTEGER, PRIMARY KEY (course_id, student_id));
```

240 students · 10 courses · ~1100 enrollments. Every query runs on a fresh, read-only, in-memory copy, with a 0.5 s limit.

Your tasks
0 of 4 done

☐ Filter and sort: CSE students in year 3, by name +20 XP

☐ Join three tables: who is enrolled in CSL100? +20 XP

SQL lab

14 Data model

Table	Contents
users	students and instructors: name, entry number, email, script hash, role
sessions	SHA-256 of each session token, expiry
assignments	title, brief, track, rubric weights, checklist, deadline, author
submissions	one row per attempt: repository, ref, status, final score, grade, area scores, summary, priorities, learning path, commit, timing
grade_overrides	instructor adjustments: original score and grade, new score and grade, reason, instructor, time
report_artifacts	full HTML, Markdown and JSON report of every finished grading
job_events	every progress event of recent gradings (pruned after 3 days), so any replica can replay a live stream
lab_progress	completed lab tasks per student
instances, meta	replica heartbeats; one-time setup markers

The same SQL runs on SQLite (local default) and PostgreSQL (production) through a thin DB-API layer, without an ORM.

15 Deployment and operations

- **Image:** multi-stage Docker build, pinned base images, dependencies installed before source for layer caching, non-root user, health check, `exec-form` command.
- **Production:** Render web service (free plan, Singapore, auto-deploys every push to main) with a Neon PostgreSQL database (free, Singapore). Reports are stored in the database because the free plan's disk is wiped on restart; the service sleeps after 15 idle minutes and wakes in about a minute.
- **Public demo mode** (`AUTOGRADER_DEMO_SEED=1`): a one-click demo student login (no public instructor login), closed sign-up, and a sample class whose public repositories are graded by the real pipeline on start-up. Turn it off for a real class.
- **Local:** `docker compose up` runs Nginx, three replicas and PostgreSQL; `scripts/start.ps1` runs a single process with SQLite.
- **Configuration:** database URL, instructor account, class join code, demo mode, Docker sandbox, concurrency, model names and timeouts are environment variables.

16 API

Method and path	Purpose	Who
POST /api/auth/signup, /login, /logout	accounts; HttpOnly session cookie	anyone
GET /api/student/dashboard	everything the student dashboard shows	student
GET /api/assignments, /api/assignments/{id}	assignments with rubric and the caller's attempts	signed in
POST /api/assignments/{id}/submit	grade a repository with the assignment's rubric (202 + job id)	signed in
GET /api/jobs/{id}/events	Server-Sent Events: stage and agent progress, verdict	owner, instructor
GET /api/jobs/{id}/report[.html .md .json]	the report	owner, instructor
GET /api/instructor/overview, /students	class analytics and student list	instructor
GET /api/instructor/gradebook[.csv]	gradebook matrix (JSON) or CSV export	instructor
GET /api/instructor/assignments/{id}	assignment statistics, distribution, copying, per-student rows	instructor
GET /api/instructor/students/{id}	one student's profile	instructor
POST/DELETE /api/instructor/submissions/{id}/override	adjust a grade with a reason / restore the original	real instructor
POST /api/instructor/submissions/{id}/regrade	grade a submission again	real instructor
POST /api/instructor/assignments/{id}/regrade	re-run every student's latest submission	real instructor
POST /api/lab/sql, /api/lab/dockerfile	server-verified lab tasks	anyone (XP when signed in)
GET /api/health, /api/metrics	health and per-stage latency	anyone

Interactive OpenAPI documentation is served at /docs on every instance.

17 Limitations and future work

- A grading runs inside the replica that accepted it, so a restart fails it and the student resubmits; the PostgreSQL work queue above would make jobs survive restarts.
- `docker build` executes the student's RUN steps with network access, so the sandbox is opt-in and intended for a dedicated VM or rootless Docker.
- Copy detection compares repositories and commits; token-level similarity (as in JPlag) across submissions is the natural next step.

- The free host sleeps when idle, so the first visit after a quiet period takes about a minute.
- Further features: regrade requests from students, per-student deadline extensions, and per-student containers so each student's backend runs on the platform itself.